E-INFO 2026 • FACAPE | TURNO DA MANHÃ • 08H00–12H00

Faculdade de Ciências Aplicadas e Sociais de Petrolina

Redes.tf

Da Teoria à Prática com Terraform e Google Cloud

Um mergulho 100% prático nos fundamentos de **VPC, Sub-redes, CIDR, Firewalls, Cloud NAT e VPC Peering** construídos como código do zero ao fim.

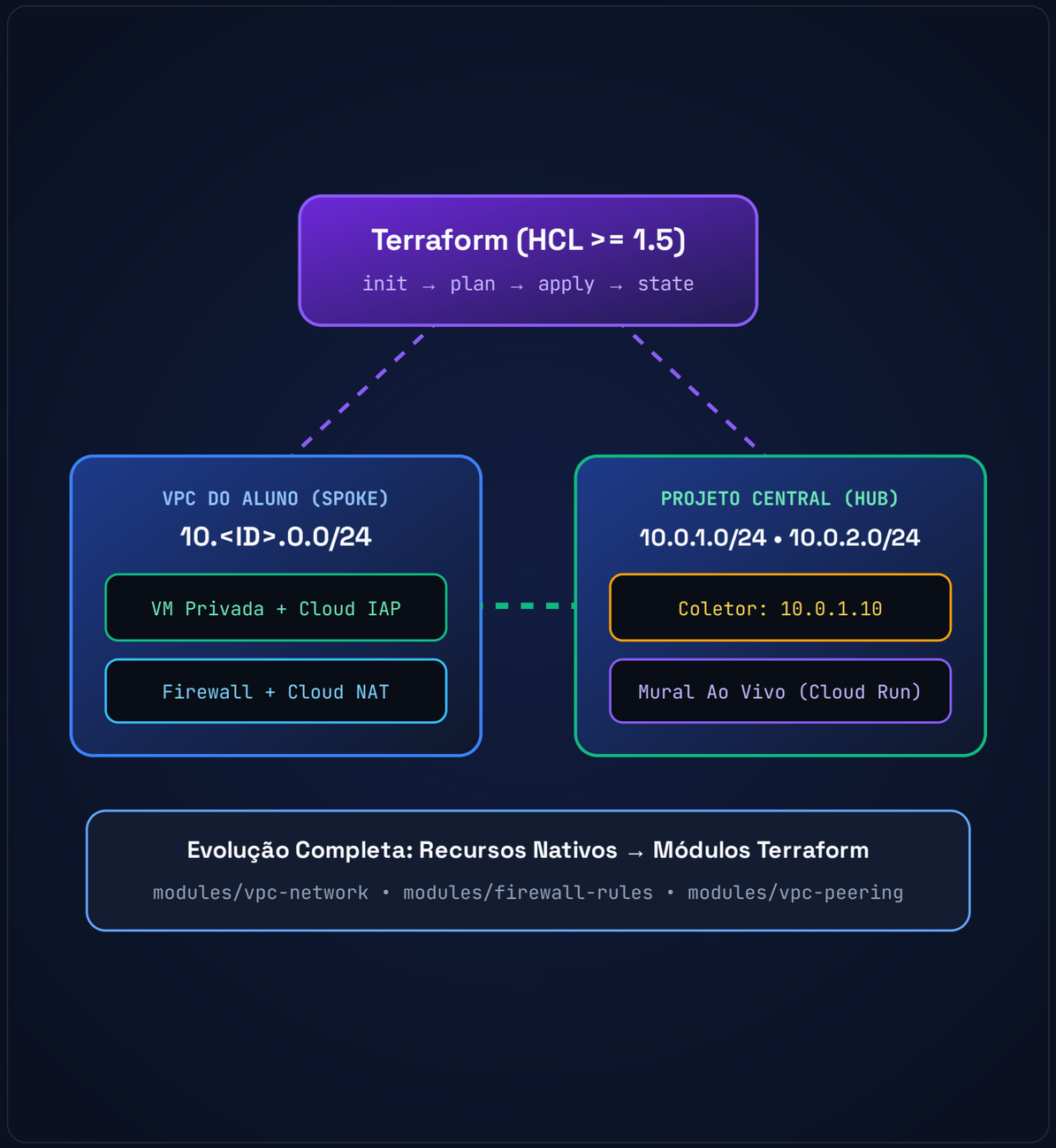


FACILITADOR

Luan Sousa

Cloud Engineer @ SantoDigital • Google Cloud Digital Leader • CC FACAPE

.tf



BOAS-VINDAS ABERTURA E METODOLOGIA

Quem vai guiar essa jornada e como vamos trabalhar



FACILITADOR DO MINICURSO

Luan Sousa

Cloud Engineer @ SantoDigital (Parceira Google Cloud)

Google Cloud Digital Leader | Ciência da Computação • FACAPE

Atua no **desenho, implantação e sustentação de arquiteturas em nuvem** no Google Cloud Platform.

- **Visão 360°:** Experiência prática passando por projetos de **Fullstack, DevOps, Infraestrutura e SecOps**.
- **Foco em IaC:** Automação de redes, segurança e ambientes reprodutíveis com Terraform.
- **Da Casa:** Bacharelando em **Ciência da Computação pela FACAPE**.

"Na faculdade vemos CIDR, roteamento, NAT e firewall na teoria. Hoje você vai ver exatamente onde cada um desses conceitos vive dentro do Google Cloud!"

01 Regra de Ouro: Zero ClickOps

Hoje nós **não vamos criar nada clicando em formulários** do Console GCP. O painel web servirá apenas para **inspecionar e comprovar visualmente** o que o nosso código Terraform construiu.

02 Construção do Zero ao Fim (Live Coding)

Vamos digitar cada bloco juntos em `terraform/live-coding/`. E se alguém atrasar? Todas as etapas já estão prontas em `terraform/labs/` como **checkpoint de segurança**!

03 Do Básico às Boas Práticas de Engenharia

Começamos com recursos explícitos para entender cada detalhe de rede sem "caixa-preta" e fechamos o dia com **Módulos Reutilizáveis do Terraform**.

CRONOGRAMA 8H VISÃO GERAL DO DIA

Nossa Trilha de Construção: Manhã vs. Tarde

TURNO 01 • 08H00 ÀS 12H00

Manhã: Fundação, VPC, Firewall e VMs

M1 **08h00–09h00 • Redes na Nuvem:** IPv4, máscara CIDR `/24`, IPs reservados e SDN Andromeda no GCP.

M2 **09h00–09h50 • Terraform do Zero:** Providers, State e setup inicial (`versions.tf`, `variables.tf`).

M3 **10h10–11h00 • Lab 01 (VPC & Sub-redes):** Criando a VPC Custom e a sub-rede `10.<ID>.0.0/24`.

M4 **11h00–12h00 • Lab 02 (Firewall, IAP & VM):** Subindo VM 100% privada, túnel SSH via IAP e o mistério do `ping 8.8.8.8`!

TURNO 02 • 14H00 ÀS 18H00

Tarde: NAT, VPC Peering e Módulos

M5 **14h00–15h00 • Lab 03 (Roteamento & Cloud NAT):** Tabelas de rotas, SNAT e saída segura para internet.

M6 **15h00–16h05 • Lab 04 (VPC Peering + Mural):** Conectando a rede do aluno ao Hub Central e check-in ao vivo no telão!

M7 **16h25–17h30 • Lab 05 (Boas Práticas — Modules):** Como organizar tudo em módulos reutilizáveis de produção.

M8 **17h30–18h00 • FinOps & Encerramento:** Grafo reverso com `terraform destroy` e carreira Cloud.

ALINHAMENTO INICIAL SEU PASSAPORTE DE REDE

Check-in do Ambiente: Seu `student_id` e `project_id`

ID Por que cada aluno tem um `student_id` único (1 a 40)?

No turno da tarde, todas as redes da sala farão **VPC Network Peering** com o Projeto Central. Em redes, **duas redes conectadas nunca podem ter o mesmo bloco de IPs (CIDR Overlap)**!

PARTICIPANTE	SUFIXO	SUB-REDE EXCLUSIVA	HUB DE DESTINO
Aluno 01	`aluno-01`	`10.1.0.0/24`	`vpc-hub-turma-a` (`10.0.1.0/24`)
Aluno 07	`aluno-07`	`10.7.0.0/24`	`vpc-hub-turma-a` (`10.0.1.0/24`)
Aluno 25	`aluno-25`	`10.25.0.0/24`	`vpc-hub-turma-b` (`10.0.2.0/24`)

Isolamento em Projetos de Grupo: Até 4 alunos compartilham um projeto GCP (`einfo-redes-grupo-XX`). Como seu Terraform usa sempre o sufixo `aluno-XX` e seu próprio `terraform.tfstate` , cada aluno gerencia apenas a própria rede — o IAM não separa colegas do mesmo projeto, então combine: **só mexa nos recursos `aluno-XX` com o seu número**.

Terminal — Verificação Rápida Copiar

```
# 1. Verifique se o Terraform está instalado
terraform -version

# 2. Confira seu projeto ativo no gcloud
gcloud config get-value project

# 3. Autentique o ADC (usado pelo Terraform!)
gcloud auth application-default login

# 4. Entre na pasta onde vamos codar do zero
cd terraform/live-coding
```

Onde foram parar os cabos, switches e roteadores na Nuvem?

LAN Rede Tradicional (On-Premises)

- **Hardware Dedicado:** Switches L2, roteadores L3, appliances físicos de Firewall e cabeamento estruturado.
- **Mudança Lenta e Manual:** Criar uma nova VLAN ou alterar uma rota exige acesso via cabo console/SSH em cada equipamento.
- **Ponto Único de Gargalo:** Todo o tráfego de saída ou firewall passa por uma "caixa física" central que pode saturar.
- **Escopo Local:** Uma LAN fica presa a um prédio ou datacenter; conectar dois estados exige contratar links MPLS/VPN caros.

SDN Google Cloud Andromeda (SDN)

- **Rede Definida por Software:** Tabelas de roteamento, sub-redes, NAT e Firewalls são **regras de software declaradas via API**.
- **Virtualização no Hipervisor:** O firewall e o roteamento acontecem direto na placa de rede virtual (vNIC) de cada máquina host, sem "caixa central" de gargalo!
- **Backbone Privado Global:** Os datacenters da Google são interligados por cabos submarinos e terrestres próprios.
- **Provisionamento em Segundos:** Com 15 linhas de Terraform, criamos uma rede global pronta para produção.

MÓDULO 01 • FUNDAMENTOS IPV4 DESMISTIFICANDO O CIDR

Entendendo o Bloco `10.<student_id>.0.0/24` na Prática

Um endereço **IPv4 possui 32 bits** divididos em **4 octetos de 8 bits** (cada número vai de `0` a `255`). A barra `/24` diz quantos bits da esquerda para a direita estão **travados para identificar a Rede**:

10	<ID>	0	0..255
00001010 (8 bits)	Aluno 01 a 40 (+8 = 16 bits)	00000000 (+8 = 24 bits)	XXXXXXXX (8 bits livres!)
1º Octeto • Rede Privada (RFC 1918)	2º Octeto • ID do Aluno	3º Octeto • Sub-rede Principal	4º Octeto • Hosts (2^8 = 256 IPs)

Analogia do CEP

Os 24 bits iniciais (`10.7.0`) são o **nome da rua**. Os 8 bits finais (`.0` a `.255`) são o **número de cada casa** naquela rua — no GCP, as VMs moram do `.2` ao `.253`.

E se fosse /16?

Em um `/16`, apenas os 2 primeiros octetos seriam fixos e sobrariam 16 bits para máquinas (**2^{16} = 65.536 IPs**).

E o /32 no Firewall?

Um `/32` trava todos os 32 bits (`2⁰ = 1 IP`). Usamos `/32` quando queremos apontar para **1 único IP exato**!

MÓDULO 01 • PEGADINHA CLÁSSICA DE CLOUD 256 - 4 = 252 HOSTS

Por que nossa primeira VM no GCP recebe o IP `.2` e não o `.1`?

Os 4 Endereços Reservados pelo Google Cloud em toda Sub-rede

ENDEREÇO (EX: ALUNO 07)	QUEM USA?	EXPLICAÇÃO TÉCNICA
`10.7.0.0`	**Endereço da Rede**	Identifica a própria sub-rede nas tabelas de roteamento.
`10.7.0.1`	**Default Gateway Virtual**	Roteador virtual da SDN Andromeda para onde a VM envia pacotes de saída.
`10.7.0.2` até `.253`	**IPs Livres (252 hosts)**	Faixa atribuída às VMs! Por isso sua 1ª VM sempre nasce com final `.2` !
`10.7.0.254`	**Penúltimo IP (Reservado)**	Reservado internamente pela Google para serviços da SDN.
`10.7.0.255`	**Endereço de Broadcast**	Último endereço do bloco `/24` segundo o padrão RFC.

FÓRMULA DE SUB-REDE NO GCP

$$2^{(32 - CIDR)} - 4$$

Para uma máscara `/24`:
$2^8 - 4 = 256 - 4 = 252$ IPs utilizáveis.

No Lab 01 vamos exibir o `gateway_address` no output do Terraform e comprovar que ele é o `10.<ID>.0.1` !

Escopo de Recursos: VPC é Global, Sub-rede é Regional, VM é Zonal

1 VPC Network = Recurso GLOBAL

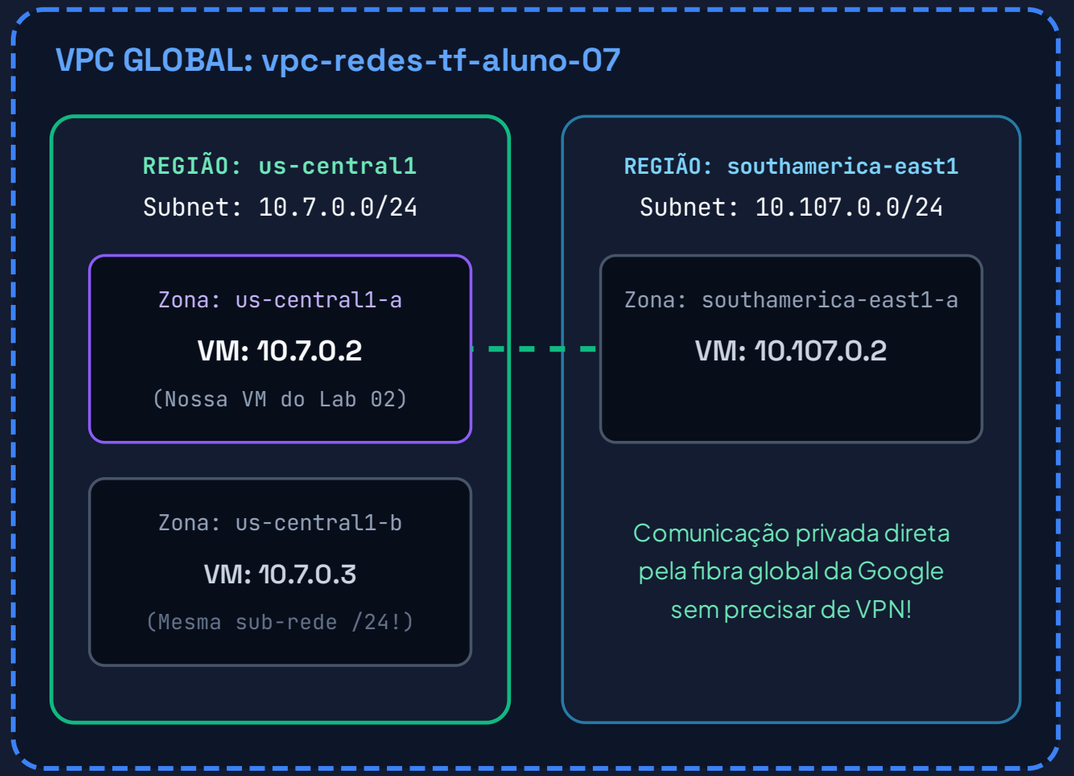
Uma única VPC no GCP atravessa o planeta inteiro. Ela não tem "região" nem faixa de IP própria: ela é o **contêiner global de roteamento e firewall**.

2 Subnetwork (Sub-rede) = Recurso REGIONAL

Cada sub-rede vive em uma região geográfica (ex: `us-central1` em Iowa ou `southamerica-east1` em São Paulo) e possui o bloco CIDR (`10.7.0.0/24`).

3 Máquina Virtual (VM) = Recurso ZONAL

A VM nasce em um datacenter específico dentro da região (ex: zona `us-central1-a`), mas consome um IP da sub-rede regional!



Por que gerenciar Redes com Terraform em vez do Console Web?

01 O Problema do ClickOps

- Passos manuais esquecidos ao recriar redes em `dev`, `staging` e `prod`.
- Regras de firewall temporárias ("só para testar") que ficam abertas para sempre sem documentação.
- Impossível fazer **Code Review (Pull Request)** de cliques no navegador.

02 O Poder Declarativo (HCL)

- Você declara **"o estado final que deseja"** (ex: quero 1 VPC e 2 regras de firewall), e não o passo a passo imperativo.
- O Terraform descobre sozinho a **ordem de criação (Grafo de Dependências)**: primeiro a VPC, depois a Sub-rede, depois a VM!

03 O Arquivo de Estado (`.tfstate`)

- O `terraform.tfstate` é a **memória do Terraform**: ele mapeia cada bloco do seu código `.tf` para o ID real do recurso criado lá no Google Cloud.
- No `terraform plan`, ele compara seu código com o `.tfstate` e mostra exatamente o que mudará!

MÓDULO 02 • SINTAXE HCL OS 5 BLOCOS E OS 5 COMANDOS

Anatomia do Terraform que vamos codar hoje

Sintaxe de um Recurso em HCL

Copiar

```hcl
# Bloco: resource "<TIPO_DO_RECURSO>" "<NOME_LOCAL>"
resource "google_compute_network" "vpc_aluno" {
  name                    = "vpc-redes-tf-aluno-07"
  auto_create_subnetworks = false
}

# Referenciando outro recurso (Grafo de Dependência!):
resource "google_compute_subnetwork" "subnet" {
  name          = "subnet-us-central1-aluno-07"
  ip_cidr_range = "10.7.0.0/24"
  network       = google_compute_network.vpc_aluno.id
}
```

1. `terraform init` — Inicializa o diretório e baixa o plugin do provider `hashicorp/google`.
2. `terraform fmt && terraform validate` — Formata o código automaticamente e valida se a sintaxe e tipos estão corretos.
3. `terraform plan` — Simula a execução e exibe o que será criado (`+`), alterado (`~`) ou removido (`-`).
4. `terraform apply` — Chama a API do Google Cloud, constrói a infraestrutura real e atualiza o `terraform.tfstate`.
5. `terraform destroy` — Remove todos os recursos na ordem inversa de dependência no final do dia.

MÃO NA MASSA • LIVE CODING 00 09H20 - 09H50

Preparando nossa Fundação em `terraform/live-coding/`

▶ Roteiro de Construção

1. Criar `versions.tf` definindo Terraform `≥ 1.5.0` e provider `hashicorp/google ⇝ 5.0`.
2. Criar `providers.tf` conectando ao `var.project_id` e `var.region`.
3. Criar `variables.tf` com validação de `student_id` entre `1` e `40`.
4. Criar `terraform.tfvars` com seu ID e rodar `terraform init`!

☕ Meta antes do Coffee Break (09h50):

Ver a mensagem **"Terraform has been successfully initialized!"** no terminal.

terraform/live-coding/versions.tf & providers.tf — Copiar

```hcl
# versions.tf
terraform {
  required_version = "≥ 1.5.0"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "⇝ 5.0"
    }
  }
}

# providers.tf
provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}
```

09h50 - 10h10

Coffee Break ☕

Antes de sair: confira se o seu `terraform init` terminou com sucesso. Se não, chame o facilitador!

```
Na volta: Etapa 01 — VPC Customizada e Sub-rede (10h10)
```

MÓDULO 03 • ETAPA 01 DECISÃO DE ARQUITETURA

Modo Auto vs. Modo Custom: Por que `auto_create_subnetworks = false`?

✕ VPC em Modo Automático (`true`)

- Cria automaticamente **1 sub-rede `/20` em cada uma das +40 regiões do mundo** dentro do bloco fixo `10.128.0.0/9`.
- **Fatal para VPC Peering:** Como todas as VPCs em modo Auto usam exatamente os mesmos IPs (`10.128.0.0/20` em `us-central1`), **duas VPCs em modo Auto nunca conseguem fazer Peering entre si**!
- Gera dezenas de sub-redes desnecessárias em regiões onde você nunca vai operar.

✓ VPC em Modo Customizado (`false`)

- A VPC nasce **100% limpa**, sem nenhuma sub-rede pré-criada.
- **Controle Total de Engenharia:** Você escolhe apenas as regiões que vai usar (`us-central1`) e define seu próprio bloco CIDR planejado (`10.<student_id>.0.0/24`).
- **Pronta para Interconexão:** Garante que não haverá sobreposição de IPs com o Hub Central nem com VPNs on-premises.
- Habilitamos `private_ip_google_access = true` na sub-rede para acesso privado às APIs da Google.

MÃO NA MASSA • ETAPA 01 10H20 – 11H00 • CHECKPOINT: LABS/01-VPC-SUBNETS/

Construindo nossa VPC Customizada e Sub-rede Regional

01 O que vamos codar agora

1. **locals.tf**: Calcular `student_suffix` e `subnet_cidr = "10.${var.student_id}.0.0/24"`.
2. **vpc.tf**: Criar `google_compute_network` e `google_compute_subnetwork` vinculada por referência implícita.
3. **outputs.tf**: Exibir o CIDR e o `gateway_address` (`10.<ID>.0.1`).
4. Executar `terraform plan` e `terraform apply` !

terraform/live-coding/vpc.tf

Copiar

```
resource "google_compute_network" "vpc_aluno" {
  name                    = local.vpc_name
  auto_create_subnetworks = false
  routing_mode            = "REGIONAL"
  description             = "VPC Custom do ${local.student_suffix} (e-info FACAPE)"
}

resource "google_compute_subnetwork" "subnet_principal" {
  name                     = local.subnet_name
  ip_cidr_range            = local.subnet_cidr
  region                   = var.region
  network                  = google_compute_network.vpc_aluno.id
  private_ip_google_access = true
}
```

Como funciona o Firewall Stateful e as Network Tags no GCP

01 Regras Implícitas da VPC

Toda VPC nasce com 2 regras implícitas de prioridade **mais baixa** possível (`65535` — qualquer regra sua vence):

- **Deny All Ingress:** Ninguém entra em nenhuma VM se você não criar uma regra `ALLOW` explícita.
- **Allow All Egress:** As VMs têm permissão para iniciar conexões de saída.

02 O que é ”Stateful”?

Em um firewall **Stateless** (antigo), você precisava liberar a entrada E também a saída da resposta.

No GCP, o firewall é **Stateful**: ele guarda uma tabela de conexões ativas (*conntrack*). Se o pacote de entrada foi permitido, **o pacote de resposta volta automaticamente**!

03 Microsegmentação com Tags

Em vez de liberar a porta `22` (SSH) para a VPC inteira, usamos `target_tags = ["ssh-iap"]` na regra de firewall.

A regra só se aplica às VMs que tiverem a etiqueta `tags = ["ssh-iap"]` declarada no Terraform!

MÓDULO 04 • ARQUITETURA ZERO-TRUST SEM IP PÚBLICO, SEM PORTA EXPOSTA

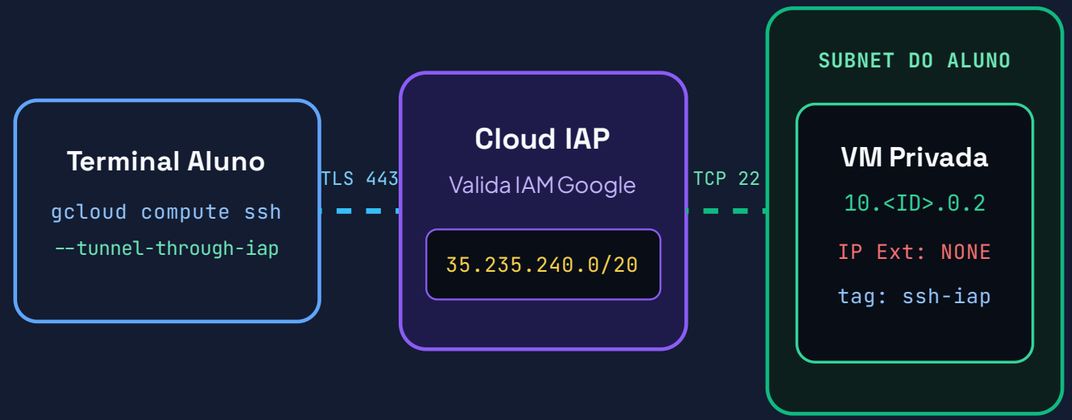
Como fazer SSH em uma VM Privada usando Cloud IAP?

! O perigo do IP Público + `0.0.0.0/0` na porta 22

Colocar um IP público numa VM e abrir a porta `22` para `0.0.0.0/0` faz com que ela receba milhares de tentativas de força bruta de bots em poucos minutos.

⎚ A Solução Profissional: Cloud IAP TCP Forwarding

- Nossa VM nasce **sem bloco** `access_config` (sem IP externo!).
- O **Identity-Aware Proxy (IAP)** da Google autentica seu usuário via IAM/HTTPS e entrega seu SSH por dentro da rede interna da Google usando o bloco exclusivo `35.235.240.0/20`.
- No Firewall, liberamos a porta `22` **apenas** para `35.235.240.0/20`!



MÃO NA MASSA • ETAPA 02 11H20 – 12H00 • CHECKPOINT: LABS/02-FIREWALL-VMS/

Subindo Firewall + VM Privada e o Mistério antes do Almoço!

02 Passo a Passo do Lab 02

1. Criar `firewall.tf` liberando `tcp:22` para `35.235.240.0/20` (tag `ssh-iap`) e `icmp` só da própria sub-rede. Nada para o Hub: **stateful**!
2. Criar `compute.tf` provisionando a VM `e2-micro` **sem bloco** `access_config`.
3. Rodar `terraform apply` e entrar via SSH usando o comando gerado no output!

? O Desafio das 11h50 (Dentro da VM):

Conectou no SSH da sua VM privada? Rode `ip a` (veja seu IP `10.<ID>.0.2`) e depois tente rodar `ping -c 3 8.8.8.8`.
Por que dá `100% packet loss`? Resolveremos às 14h00!

terraform/live-coding/firewall.tf & compute.tf — Copiar

```hcl
resource "google_compute_firewall" "allow_ssh_iap" {
  name          = "fw-allow-ssh-iap-${local.student_suffix}"
  network       = google_compute_network.vpc_aluno.id
  direction     = "INGRESS"
  source_ranges = ["35.235.240.0/20"]
  target_tags   = ["ssh-iap"]
  allow {
    protocol = "tcp"
    ports    = ["22"]
  }
}

resource "google_compute_instance" "vm_aluno" {
  name         = "vm-teste-${local.student_suffix}"
  machine_type = "e2-micro"
  zone         = var.zone
  tags         = ["ssh-iap", local.student_suffix]
  boot_disk {
    initialize_params { image = "debian-cloud/debian-12" }
  }
  network_interface {
    subnetwork = google_compute_subnetwork.subnet_principal.id
  }
}
```

e-info 2026 | FACAPE

12h00 – 14h00

Almoço

Deixe sua VM ligada: o mistério do `100% packet loss` será resolvido na volta.

```
Na volta: Etapa 03 — Tabela de Rotas e Cloud NAT (14h00)
```